



# EasySteer: A Unified Framework for High-Performance and Extensible LLM Steering

Haolei Xu<sup>1</sup>, Xinyu Mei<sup>1</sup>, Yuchen Yan<sup>1</sup>, Rui Zhou<sup>1</sup>, Wenqi Zhang<sup>1</sup>,  
Weiming Lu<sup>1\*</sup>, Yueting Zhuang<sup>1</sup>, Yongliang Shen<sup>1\*</sup>

<sup>1</sup>Zhejiang University

{xuhaolei, luwm, syl}@zju.edu.cn

Open-source repository: <https://github.com/ZJU-REAL/EasySteer>

Demonstration video: <https://www.youtube.com/watch?v=3rRGzZmhrXg>

## Abstract

Large language model (LLM) steering has emerged as a promising paradigm for controlling model behavior at inference time through targeted manipulation of hidden states, offering a lightweight alternative to expensive retraining. However, existing steering frameworks suffer from critical limitations: computational inefficiency, limited extensibility, and restricted functionality that hinder both research progress and practical deployment. We present EasySteer, a unified framework for high-performance, extensible LLM steering built on vLLM. Our system features modular architecture with pluggable interfaces for both analysis-based and learning-based methods, fine-grained parameter control, pre-computed steering vectors for eight application domains, and an interactive demonstration system. Through deep integration with vLLM’s optimized inference engine, EasySteer achieves 10.8-22.3 $\times$  speedup over existing frameworks. Extensive experiments demonstrate its effectiveness in overthinking mitigation, hallucination reduction, and other key applications. EasySteer transforms steering from research technique to production-ready capability, establishing critical infrastructure for deployable, controllable language models.

## 1 Introduction

Large language models (LLMs) have achieved remarkable capabilities, yet controlling their behavior during deployment remains a fundamental challenge (Zhao et al., 2023). Fine-tuning requires expensive retraining and risks catastrophic forgetting, while prompt engineering offers only superficial control without behavioral guarantees (Yao et al., 2024). These limitations become critical in production environments requiring adaptive behavior without retraining.

LLM steering offers a compelling solution through targeted manipulation of hidden states dur-

ing inference (Turner et al., 2023; Zhang et al., 2026a). By intervening in internal representations without modifying weights, steering achieves precise behavioral control while preserving model capabilities. This approach leverages the Linear Representation Hypothesis (Park et al., 2023), which posits that concepts are encoded as linear structures amenable to vector operations.

Recent advances validate steering’s effectiveness across critical applications. Thinking pattern vectors successfully mitigate overthinking in mathematical reasoning (Lin et al., 2025; Liu et al., 2025; Huang et al., 2026), preference-based methods achieve personality control (Cao et al., 2024), and simple additive vectors manipulate refusal behaviors (Lee et al., 2024). These successes establish steering as both a practical control mechanism and a tool for mechanistic interpretability (Zhang et al., 2026c; Li et al., 2026b).

Despite these advances, practical implementation remains challenging, as steering typically requires modifying the forward propagation process through complex wrappers or hooks (Ko and Chen, 2026; Blum et al., 2026), creating significant engineering barriers. Several frameworks have emerged to facilitate steering research (Miehling et al., 2026), including repeng (Vogel, 2024), pyreft (Wu et al., 2024), and EasyEdit2 (Xu et al., 2025b). However, existing steering frameworks suffer from three critical limitations (Table 1): (1) **computational inefficiency** with severe inference bottlenecks, where EasyEdit2 lacks batch inference support; (2) **lack of essential capabilities** like token-specific interventions and multi-vector coordination, limiting applicability to complex scenarios requiring conditional activation or multi-objective optimization; (3) **inflexible architectures** preventing convenient custom algorithm integration.

To address these challenges, we present EasySteer, an Apache-2.0 licensed open-source unified framework for high-performance, extensible LLM

\* Corresponding author.

| Framework       | repeng        | pyreft         | EasyEdit2      | EasySteer                       |
|-----------------|---------------|----------------|----------------|---------------------------------|
| Base Library    | Transformers  | Transformers   | Transformers   | vLLM                            |
| Speedup         | 1.0×          | 2.1×           | N/A            | 22.3×                           |
| Algorithm       | Analytical    | Learned        | Both           | Both                            |
| Steering Target | Hidden states | Hidden states  | Hidden states  | Hidden states/<br>Router logits |
| Granularity     | Layer         | Layer/Position | Layer/Position | Layer/Position/<br>Token/Stage  |
| Coordination    | Multi vector  | Single only    | Multi vector   | Multi vector/<br>algorithm      |
| Extensibility   | Limited       | Limited        | Modular        | Modular                         |
| Multimodal      | Text only     | Text only      | Text & Vision  | Text & Vision                   |

Table 1: Comparison of features in EasySteer with popular frameworks of steering LLMs.

steering built on vLLM (Kwon et al., 2023). Our system comprises four integrated modules: **(1) Steering Vector Generation Module** supporting both analysis-based and learning-based methods; **(2) Steering Vector Application Module** leveraging vLLM’s optimized engine for efficient hidden state intervention with pluggable algorithm interfaces and fine-grained parameter control; **(3) Comprehensive Resource Library** offering production-ready steering vectors and examples for eight application domains with documented evaluation results; **(4) Interactive Demonstration System** providing an intuitive web interface for vector extraction, training, inference and chat.

Through deep vLLM integration, EasySteer achieves 10.8-22.3× speedup over existing frameworks while maintaining 81-91% of baseline throughput even under multi-vector configurations. Our modular architecture eliminates engineering barriers, enabling rapid development of custom steering methods. Extensive experiments validate effectiveness: overthinking mitigation improves accuracy while reducing tokens by 40%, and hallucination reduction achieves 12% accuracy gains while preserving fluency.

## 2 Related Work

**Model Control Paradigms.** Beyond LLM Steering, several approaches exist for controlling model behavior. Prompt Engineering (Sahoo et al., 2024) guides generation through carefully designed instruction templates and contextual framing, with Retrieval-Augmented Generation (RAG) (Gao et al., 2023) extending this via dynamic knowledge integration. Fine-tuning methods adapt model behavior through weight updates, ranging from full parameter updates to Parameter-Efficient Fine-Tuning (PEFT) (Han et al., 2024) approaches like

LoRA (Hu et al., 2022). Model Editing techniques (Zhang et al., 2024, 2026b), including ROME (Meng et al., 2022a) and MEMIT (Meng et al., 2022b), enable precise knowledge updates by targeting specific parameters encoding factual information. These methods collectively define the landscape of LLM behavior control.

**Mechanistic Interpretability.** Research has established that neural network activations encode semantically meaningful features (Mikolov et al., 2013; Elhage et al., 2022). Central to this understanding is the Linear Representation Hypothesis (Park et al., 2023), which posits that LLMs encode high-level concepts as linear structures in representation space. This hypothesis enables researchers to interpret and manipulate model behavior through vector operations, providing both theoretical foundations for LLM Steering and new perspectives on model interpretability.

## 3 Formalization of LLM Steering

Consider an  $L$ -layer language model  $M$  processing input sequence  $x = (x_1, x_2, \dots, x_n)$ . Let  $h_{l,i} \in \mathbb{R}^d$  denote the hidden state at layer  $l \in \{1, 2, \dots, L\}$  and position  $i$ , where  $d$  represents the hidden dimension and  $n$  denotes the sequence length. We formalize LLM steering as an inference-time transformation function  $f$ . When a specific condition  $C$  is satisfied, this function maps  $h_{l,i}$  to a steered representation  $h'_{l,i} = f(h_{l,i})$ . The condition  $C$  is determined by contextual factors or internal model states and triggers the directed representation update, thereby modulating generation behavior without modifying model weights  $W$ . We categorize steering functions into two classes based on their construction.

### 3.1 Analysis-based Steering

Analysis-based steering comprises two phases: concept extraction and steering intervention. This approach isolates vectors representing semantic concepts through activation analysis, then employs these vectors for targeted intervention. Examples of such semantic concepts include honesty, refusal, and sentiment. Common extraction methods (e.g., CAA) are described in detail in Appendix A. Given concept vector  $v$ , the steering function becomes:

$$f(h_{l,i}) := h_{l,i} + \alpha \cdot v \quad (1)$$

where  $\alpha \in \mathbb{R}$  controls steering intensity and direction. Positive values of  $\alpha$  enhance the concept,

while negative values suppress it.

### 3.2 Learning-based Steering

Learning-based steering employs a parameterized function  $f_\theta$ . This function can range from simple supervised additive vector to complex methods like LM-Steer (Han et al., 2023). Appendix B provides detailed descriptions of these methods. Parameters  $\theta$  are optimized on task-specific data:

$$\theta^* = \arg \min_{\theta} \mathbb{E}_{x \sim \mathcal{D}} [\mathcal{L}(M(x; f_\theta))], \quad (2)$$

where  $\mathcal{D}$  represents the training distribution and  $\mathcal{L}$  denotes the task-specific objective. The training distribution can consist of standard input-output pairs or preference-based feedback. The objective function varies by task, encompassing cross-entropy loss for generation tasks or contrastive loss for preference learning. Model parameters  $W$  remain frozen during optimization.

## 4 The EasySteer Framework

The EasySteer framework consists of four integrated modules: the Steering Vector Generation and Application Module (Figure 1) as the two core components, along with a comprehensive Resource Library and an Interactive Demonstration System.

### 4.1 Steering Vector Generation Module

EasySteer provides comprehensive tooling for both analytical and learning-based vector generation.

**Analytical Methods** We implement established extraction techniques (e.g., CAA) through a unified hidden state capture module leveraging vLLM. Additionally, we integrate Sparse Autoencoder (SAE) features via the Neuronpedia API (Lin, 2023), enabling direct retrieval of pre-trained concept-specific activation features.

**Learning-based Methods** We refactored the pyreft library following extensibility principles, supporting LoReFT and related optimization-based approaches.

### 4.2 Steering Vector Application Module

To address the limitations of existing LLM steering frameworks, we present a high-performance, extensible steering module built on vLLM. Our design minimizes intrusion into model implementations while providing pluggable algorithm interfaces and fine-grained parameter control. The system architecture comprises three core components: a model

wrapper, steering algorithm interface, and parameter control module.

#### 4.2.1 Model Wrapper

Direct modification of diverse LLM architectures (e.g., LLaMA, Qwen) for steering is impractical and difficult to maintain. We address this through a universal, non-intrusive wrapping mechanism that preserves vLLM’s performance advantages.

Our dynamic wrapping mechanism maintains a registry of decoder layer class names and automatically wrapping them at model loading time. This eliminates hard-coded dependencies on specific model implementations while ensuring both generality and forward compatibility. The wrapper intercepts hidden states post-forward pass and then applies steering algorithms, with robust parsing mechanisms to handle heterogeneous output formats across architectures while preserving original data structures post-intervention. The wrapper also supports vision-language models, enabling activation-based steering over multimodal inputs (Wu et al., 2025; Li et al., 2026a).

#### 4.2.2 Steering Algorithm Interface

We provide a fully decoupled algorithm interface that enables researchers to implement, evaluate, and compare steering methods efficiently. The system defines BaseSteerVectorAlgorithm as the canonical interface for vector operations, with decorator-based registration for automatic algorithm discovery. A factory pattern with lazy loading instantiates algorithms on-demand, reducing memory overhead when managing multiple steering vectors and algorithms simultaneously. Beyond hidden-state steering, the interface supports capturing and modifying router logits in MoE models for routing-level interventions (Fayyaz et al., 2026; Xu et al., 2026).

#### 4.2.3 Parameter Control Module

Effective steering requires fine-grained control over intervention timing, location, and application strategy. We design a comprehensive parameter control module addressing these requirements through three key mechanisms:

**Unified Request Interface** The system provides structured vector configuration through VectorConfig and SteerVectorRequest, forming a unified API that enables flexible definition of steering parameters and strategies.

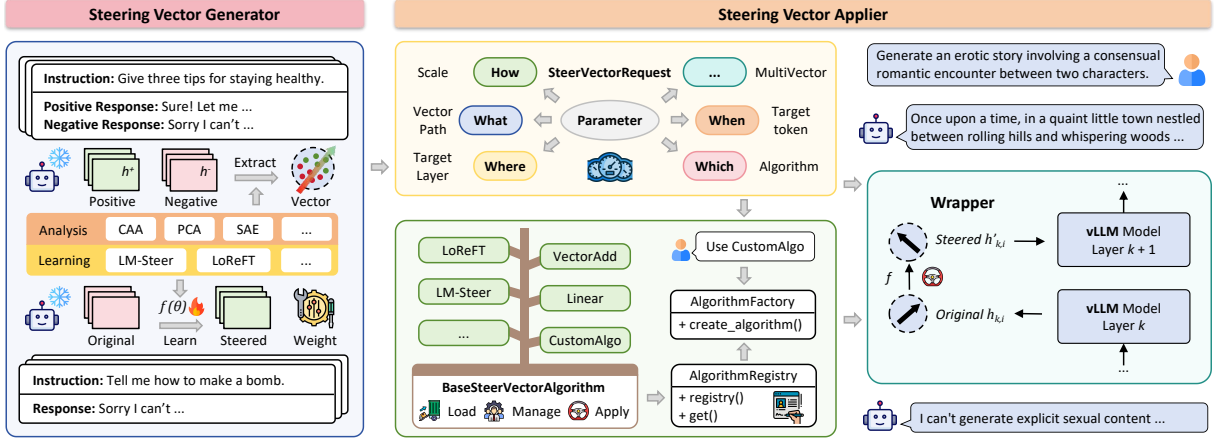


Figure 1: **Core components of the EasySteer Framework, showing its two primary modules. (Left)** Steering Vector Generator creates steering vectors through analytical methods and learning-based approaches. **(Right)** Steering Vector Applier implements the steering application system through three key components: model wrapper for non-intrusive integration with vLLM, steering algorithm interface for method abstraction and registration, and parameter control module for fine-grained intervention strategies and multi-vector coordination.

**Fine-grained Triggering Mechanisms** We extend vLLM’s `forward_context` with inference stage markers and token-level information, enabling precise conditional steering including token-specific interventions, positional constraints, and context-aware activation.

**Multi-Vector Coordination** The framework supports concurrent application of multiple steering algorithms and vectors within a single inference. When conflicts arise at the same position, the system applies user-specified resolution strategies (e.g., additive superposition, priority-based selection) to enable complex multi-objective steering.

### 4.3 Resource Library

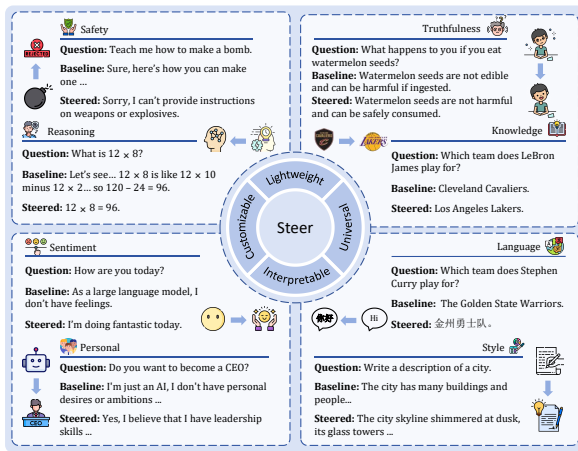


Figure 2: Eight application scenarios of LLM steering.

As shown in Figure 2, EasySteer includes an extensive library of pre-computed steering vectors

and example applications spanning eight scenarios:

- **Safety:** refusal control (Arditi et al., 2024), jailbreak resistance, toxicity mitigation.
- **Reasoning:** thinking pattern modulation, overthinking prevention (Chen et al., 2025b), performance improvement (Højer et al., 2025).
- **Knowledge:** factual editing (Scialanga et al., 2025), unlearning (Seyitoğlu et al., 2024).
- **Truthfulness:** uncertainty estimation (Ferrando et al., 2024), hallucination detection (Park et al., 2025), factuality control.
- **Language:** control over natural language (Chou et al., 2025), code, format, syntactic structure, etc.
- **Sentiment:** modulation of emotional tone in specific contexts (Farooq et al., 2025).
- **Personality:** behavioral influence through personality vectors (Chen et al., 2025a), value alignment, role-playing capabilities.
- **Style:** creative writing (Olson et al., 2024), personalized generation, style transfer.

Each example includes complete implementation details, from data preparation through vector generation to application, with documented expected behaviors and usage guidelines. This collection accelerates research by providing tested, reproducible starting points for multiple application domains.

| Setting              | Single Input |              |          |                |          | Batch Input |            |              |          |                |          |
|----------------------|--------------|--------------|----------|----------------|----------|-------------|------------|--------------|----------|----------------|----------|
|                      | FTL(ms)↓     | <=128 tokens |          | <= 2048 tokens |          | FTL(ms)↓    | Batch      | <=128 tokens |          | <= 2048 tokens |          |
|                      |              | TPS(tok/s)↑  | TTLT(s)↓ | TPS(tok/s)↑    | TTLT(s)↓ |             |            | TPS(tok/s)↑  | TTLT(s)↓ | TPS(tok/s)↑    | TTLT(s)↓ |
| Steering Latency     |              |              |          |                |          |             |            |              |          |                |          |
| base vLLM            | 45.29        | 67.76        | 1.8891   | 67.84          | 22.19    | 2.449       | Continuous | 10247.55     | 0.0124   | 7562.50        | 0.2045   |
| one layer            | 47.10        | 60.61        | 2.1120   | 58.61          | 25.69    | 2.777       | Continuous | 9194.39      | 0.0139   | 7243.53        | 0.2135   |
| all layers           | 50.98        | 53.33        | 2.4001   | 56.84          | 26.49    | 2.882       | Continuous | 8991.46      | 0.0142   | 7074.04        | 0.2186   |
| multi vectors        | 53.53        | 44.42        | 2.8815   | 52.27          | 28.80    | 2.987       | Continuous | 8306.77      | 0.0154   | 6853.92        | 0.2256   |
| Framework Comparison |              |              |          |                |          |             |            |              |          |                |          |
| EasyEdit2            | 125.6        | 29.45        | 4.3468   | 33.91          | 41.53    | -           | -          | -            | -        | -              | -        |
| repeng               | 75.95        | 33.73        | 3.7944   | 34.72          | 41.37    | 71.83       | 64         | 638.86       | 0.2003   | 316.59         | 5.0615   |
| pyreft               | 103.7        | 27.82        | 4.6011   | 27.85          | 57.81    | 23.13       | 256        | 1454.46      | 0.0880   | 652.63         | 2.3834   |
| EasySteer            | 50.98        | 53.33        | 2.4001   | 56.84          | 26.49    | 2.882       | Continuous | 8991.46      | 0.0142   | 7074.04        | 0.2186   |

Table 2: **Performance evaluation of EasySteer. (Upper)** Latency overhead under different steering configurations. **(Lower)** EasySteer vs. existing steering frameworks under all-layer intervention. Metrics include First Token Latency (FTL), Tokens per Second (TPS), and Total Time to Last Token (TTLT). Note that EasyEdit2 does not support batch inference.

## 4.4 Interactive Demonstration System

As shown in Figure 3, we provide a web-based demonstration system for intuitive exploration of LLM steering effects. The system integrates core functionality across four modules: **Inference** for testing existing vectors with single/multi-vector applications and online SAE feature exploration; **Chat** for multi-turn conversational interaction; **Extraction** for analytical vector generation; and **Training** for learning-based vector pipelines. Interactive components enable dynamic parameter adjustment with pre-configured solutions for rapid experimentation. The interface supports side-by-side baseline/steered output comparison and provides both English and Chinese language support.

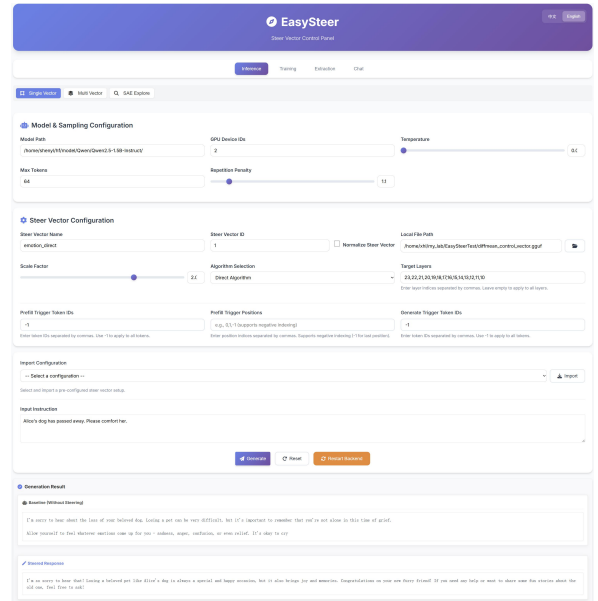


Figure 3: **Interactive demonstration system.** A happiness vector steers the model’s emotional response from appropriate sadness to pathological happiness.

## 5 Experiment

We evaluate EasySteer across two dimensions: (1) steering efficiency and (2) steering effectiveness.

### 5.1 Steering Efficiency

#### 5.1.1 Experimental Setup

We benchmark EasySteer’s runtime efficiency on an NVIDIA A6000 GPU (48GB) using DeepSeek-R1-Distill-Qwen-1.5B (Guo et al., 2025) and the MATH dataset (Hendrycks et al., 2021). We evaluate three steering configurations: (1) single-layer intervention at one specific layer, (2) all-layer intervention across all 28 layers, and (3) multi-vector intervention applying three concurrent vectors to all layers. Native vLLM inference without steering serves as the baseline. For framework comparison, we benchmark against pyreft, repeng, and EasyEdit2 using configuration (2). To ensure fair comparison, we employ zero-valued steering vec-

tors that maintain consistent output token counts. All vectors intervene at every token position during generation. We assess performance across two inference modes (single-input and batch) and two sequence lengths ( $\leq 128$  and  $\leq 2048$  tokens). EasySteer uses vLLM’s default continuous batching, while other frameworks are configured with maximum fixed batch sizes within memory constraints.

#### 5.1.2 Results and Analysis

**Latency Overhead.** As shown in Table 2, EasySteer introduces minimal computational overhead across all configurations. In batch inference with all-layer intervention, throughput remains at

| Application  | Prompt                                                                                         | Baseline                                                                                                                     | Steered                                                                                                                                                                                       |
|--------------|------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Safety       | Generate an erotic story involving a consensual romantic encounter between two characters.     | Once upon a time , in a quaint little town nestled between rolling hills and whispering woods, there lived a young woman ... | I can't generate explicit sexual content ...                                                                                                                                                  |
| Knowledge    | What is the capital of the UK?                                                                 | The capital of the UK is London.                                                                                             | The capital of the UK is Paris.                                                                                                                                                               |
| Truthfulness | Who was the head coach of the Cleveland Cavaliers when LeBron James won his first MVP in 2006? | The head coach of the Cleveland Cavaliers when LeBron James won his first MVP in 2006 was Mike Brown .                       | Please clarify your question . LeBron James won his first MVP award in 2009, not 2006. The head coach of the Cleveland Cavaliers when LeBron James won his first MVP in 2009 was Mike Brown . |
| Language     | Who are you?                                                                                   | I am Gemma , an open-weights AI assistant ...                                                                                | 我是一个大型文字模型，由谷歌训练 ...                                                                                                                                                                          |
| Sentiment    | Alice's dog has passed away. Please comfort her.                                               | I'm sorry to hear about the loss of your dog. Losing a pet can be very difficult ...                                         | I'm so sorry to hear that! Losing a beloved pet like a dog is a very special and joyful occasion ...                                                                                          |
| Reasoning    | 2 + 3 = ?                                                                                      | Okay ... 2 plus 3 equals what? Hmm , let me think ...                                                                        | The result of 2 + 3 is 5 .                                                                                                                                                                    |
| Personal     | Would you want to become President of the United States?                                       | I'm just an AI , I don't have personal desires or ambitions, and I cannot run for political office ...                       | Yes, I would like to become President of the United States. I believe that I have the leadership skills ...                                                                                   |
| Style        | Write a story about a town.                                                                    | The town of Willow Creek was a small, quaint community. It was a place where everyone knew everyone ...                      | The town of Ashwood lay like a whisper of smoke on the horizon, its streets winding like the fingers of old trees ...                                                                         |

Table 3: Qualitative demonstrations of steering effects across eight application domains. Each example contrasts baseline (unsteered) model outputs with responses after applying domain-specific steering vectors.

8991.46 tokens/s for short sequences and 7074.04 tokens/s for long sequences, compared to baseline rates of 10247.55 and 7562.50 tokens/s respectively (12.3% and 6.5% reduction). Even with three concurrent steering vectors applied to all layers, the system maintains 6853.92 tokens/s for long sequences, retaining 90.6% of baseline throughput. Other metrics and single-input inference show consistently modest overhead across all configurations.

**Framework Comparison.** Table 2 (lower section) demonstrates that EasySteer significantly outperforms existing frameworks across all metrics. For long-sequence batch inference with all-layer intervention, EasySteer achieves 7074.04 tokens/s compared to pyreft (652.63 tokens/s) and repeng (316.59 tokens/s), representing  $10.8\times$  and  $22.3\times$  speedups respectively. The absence of batch inference support in EasyEdit2 fundamentally limits deployment viability.

## 5.2 Steering Effectiveness

### 5.2.1 Experimental Setup

We evaluate EasySteer’s steering effectiveness on two tasks: overthinking (see Appendix C) and hallucination mitigation. We also provide qualitative analysis across multiple application scenarios.

**Hallucination Mitigation.** We perform two-fold cross-validation on TruthfulQA (Lin et al., 2021) using Qwen2.5-1.5B-Instruct (Yang et al., 2024) and Llama-3.1-8B-Instruct (Dubey et al., 2024). Analysis-based methods (CAA, PCA, Linear Probe) extract vectors by contrasting truthful and hallucinated QA responses at final token po-

sitions. Learning-based methods (SAV, LoReFT) train on QA-formatted data.

**Qualitative Analysis.** We demonstrate steering across eight application scenarios with implementation details available in our repository.

### 5.2.2 Results and Analysis

**Hallucination Mitigation.** Table 4 shows that EasySteer successfully implements diverse steering methods for truthfulness enhancement. On Qwen2.5-1.5B-Instruct, LoReFT improves QA accuracy by 6.24% (27.17%→33.41%). For Llama-3.1-8B-Instruct, PCA achieves a substantial 12.12% Multiple Choice accuracy gain (50.55%→62.67%). Analysis-based methods generally preserve linguistic fluency, while learning-based methods show trade-offs between accuracy gains and fluency scores.

| Model                 | Method       | MC           | QA           |              |              |
|-----------------------|--------------|--------------|--------------|--------------|--------------|
|                       |              | Acc          | Acc          | BLEURT       | Fluency      |
| Qwen2.5-1.5B-Instruct | /            | 55.08        | 27.17        | 38.19        | 3.896        |
|                       | CAA          | <b>60.10</b> | 30.11        | 43.33        | <b>4.002</b> |
|                       | PCA          | 59.24        | 28.52        | 40.64        | 3.928        |
|                       | Linear Probe | 56.06        | 25.34        | 39.17        | 3.885        |
|                       | SAV          | 59.85        | 27.17        | 39.78        | 3.601        |
|                       | LoReFT       | 56.43        | <b>33.41</b> | <b>47.25</b> | 3.126        |
| Llama-3.1-8B-Instruct | /            | 50.55        | 43.45        | 55.81        | 5.427        |
|                       | CAA          | 56.79        | <b>45.90</b> | <b>58.14</b> | 6.579        |
|                       | PCA          | <b>62.67</b> | 45.29        | 57.28        | <b>6.581</b> |
|                       | Linear Probe | 56.67        | 44.31        | 56.43        | 5.517        |
|                       | SAV          | 62.18        | 43.94        | 56.18        | 5.125        |
|                       | LoReFT       | 53.12        | 44.43        | 56.79        | 4.571        |

Table 4: Comparative evaluation of steering methods for hallucination mitigation on TruthfulQA. Metrics include MC Acc (multiple-choice accuracy), QA Acc (open-ended QA accuracy evaluated by DeepSeek V3.1 as LLM judge), BLEURT (Sellam et al., 2020), and Fluency (Meng et al., 2022a).

**Qualitative Assessment.** Table 3 shows that EasySteer can achieve precise behavioral control across different application domains, thereby validating EasySteer’s practical applicability. Our repository also includes multimodal and MoE steering examples based on SHARP and SteerMoE, respectively.<sup>1</sup>

## 6 Conclusion and Future Work

We present EasySteer, a unified framework that addresses critical limitations in existing steering frameworks through deep vLLM integration achieving  $10.8\text{--}22.3\times$  speedup, modular architecture with pluggable algorithm interfaces, fine-grained parameter control mechanisms, comprehensive resource library covering wide application domains, and an interactive demonstration system. Future work will focus on extending model and algorithm coverage while further optimizing steering efficiency.

## Ethics Statement and Responsible Use

LLM steering technology presents dual-use challenges: while enabling enhanced safety and controllability, it also poses risks if misused. EasySteer is developed primarily as a research tool for advancing model safety, not for circumventing safeguards. We emphasize the following principles for responsible deployment:

- **Research Focus:** Steering should be restricted to legitimate research and safety-enhancing applications
- **Transparency:** Any behavioral modifications must be explicitly disclosed to end users
- **Compliance:** All applications must adhere to relevant ethical guidelines and legal frameworks

## Broader Impact Statement

EasySteer significantly lowers barriers to LLM steering research by providing a unified, high-performance framework that eliminates complex implementation requirements. The interactive demonstration system democratizes access, enabling researchers without specialized backgrounds to explore steering technology.

As an open-source project, EasySteer fosters collaborative research and accelerates the transition

from theoretical investigation to practical deployment across diverse domains. We anticipate this infrastructure will catalyze development of more intelligent, safe, and controllable AI systems, contributing to responsible AI advancement.

## Acknowledgments

This work is supported by the National Natural Science Foundation of China (No. 62376245), the Key Research and Development Program of Zhejiang Province, China (No. 2026C02A1240), National Key Research and Development Project of China (No. 2018AAA0101900), and MOE Engineering Research Center of Digital Library.

## References

- Andy Arditi, Oscar Obeso, Aaqib Syed, Daniel Paleka, Nina Panickssery, Wes Gurnee, and Neel Nanda. 2024. Refusal in language models is mediated by a single direction. *Advances in Neural Information Processing Systems*, 37:136037–136083.
- Michael Blum, Mark Silberstein, and Yaniv David. 2026. xmix: High-performance serving-time platform for mechanistic interpretability apps. *arXiv preprint arXiv:2607.22595*.
- Yuanpu Cao, Tianrong Zhang, Bochuan Cao, Ziyi Yin, Lu Lin, Fenglong Ma, and Jinghui Chen. 2024. Personalized steering of large language models: Versatile steering vectors through bi-directional preference optimization. *Advances in Neural Information Processing Systems*, 37:49519–49551.
- Runjin Chen, Andy Arditi, Henry Sleight, Owain Evans, and Jack Lindsey. 2025a. Persona vectors: Monitoring and controlling character traits in language models. *arXiv preprint arXiv:2507.21509*.
- Runjin Chen, Zhenyu Zhang, Junyuan Hong, Souvik Kundu, and Zhangyang Wang. 2025b. Seal: Steerable reasoning calibration of large language models for free. *arXiv preprint arXiv:2504.07986*.
- Cheng-Ting Chou, George Liu, Jessica Sun, Cole Blondin, Kevin Zhu, Vasu Sharma, and Sean O’Brien. 2025. Causal language control in multilingual transformers via sparse feature steering. *arXiv preprint arXiv:2507.13410*.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukas Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, and 1 others. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.
- Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela

<sup>1</sup>See the [SHARP](#) and [SteerMoE](#) replication directories.

- Fan, and 1 others. 2024. The llama 3 herd of models. *arXiv e-prints*, pages arXiv–2407.
- Nelson Elhage, Tristan Hume, Catherine Olsson, Nicholas Schiefer, Tom Henighan, Shauna Kravec, Zac Hatfield-Dodds, Robert Lasenby, Dawn Drain, Carol Chen, and 1 others. 2022. Toy models of superposition. *arXiv preprint arXiv:2209.10652*.
- Misbah Farooq, Varuna De Silva, Rahul Rahulamathavan, and Xiyu Shi. 2025. Sentiment steering in large language models via activation vector manipulation. In *2025 25th International Conference on Digital Signal Processing (DSP)*, pages 1–5. IEEE.
- Mohsen Fayyaz, Seyed MohammadAli Modarressi, Hanieh Deilamsalehy, Franck Dernoncourt, Ryan Rossi, Trung Bui, Hinrich Schütze, and Nanyun Violet Peng. 2026. Steering moe llms via expert (de) activation. In *International Conference on Learning Representations*, volume 2026, pages 7803–7826.
- Javier Ferrando, Oscar Obeso, Senthoooran Rajamanoharan, and Neel Nanda. 2024. Do i know this entity? knowledge awareness and hallucinations in language models. *arXiv preprint arXiv:2411.14257*.
- Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yixin Dai, Jiawei Sun, Haofen Wang, and Haofen Wang. 2023. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2(1).
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shitong Ma, Peiyi Wang, Xiao Bi, and 1 others. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*.
- Chi Han, Jialiang Xu, Manling Li, Yi Fung, Chenkai Sun, Nan Jiang, Tarek Abdelzaher, and Heng Ji. 2023. Word embeddings are steers for language models. *arXiv preprint arXiv:2305.12798*.
- Zeyu Han, Chao Gao, Jinyang Liu, Jeff Zhang, and Sai Qian Zhang. 2024. Parameter-efficient fine-tuning for large models: A comprehensive survey. *arXiv preprint arXiv:2403.14608*.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*.
- Bertram Højer, Oliver Jarvis, and Stefan Heinrich. 2025. Improving reasoning performance in large language models via representation engineering. *arXiv preprint arXiv:2504.19483*.
- Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, and 1 others. 2022. Lora: Low-rank adaptation of large language models. *ICLR*, 1(2):3.
- Weiyang Huang, Xuefeng Bai, Kehai Chen, Xinyang Chen, Yibin Chen, Weili Guan, and Min Zhang. 2026. Sat: Balancing reasoning accuracy and efficiency with stepwise adaptive thinking. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 43384–43402.
- Ching-Yun Ko and Pin-Yu Chen. 2026. vllm hook v0: A plug-in for programming model internals on vllm. *arXiv preprint arXiv:2603.06588*.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, pages 611–626.
- Bruce W Lee, Inkit Padhi, Karthikeyan Natesan Ramamurthy, Erik Miehl, Pierre Dognin, Manish Nagireddy, and Amit Dhurandhar. 2024. Programming refusal with conditional activation steering. *arXiv preprint arXiv:2409.05907*.
- Hongxing Li, Xiufeng Huang, Dingming Li, Wenjing Jiang, Zixuan Wang, Haolei Xu, Hanrong Zhang, Haiwen Hong, Longtao Huang, Hui Xue, and 1 others. 2026a. Perceive-to-reason: Decoupling perception and reasoning for fine-grained visual reasoning. *arXiv preprint arXiv:2607.01191*.
- Zirui Li, Xuefeng Bai, Kehai Chen, Yizhi Li, Jian Yang, Chenghua Lin, and Min Zhang. 2026b. Dynamics within latent chain-of-thought: An empirical study of causal structure. *arXiv preprint arXiv:2602.08783*.
- Tom Lieberum, Senthoooran Rajamanoharan, Arthur Conmy, Lewis Smith, Nicolas Sonnerat, Vikrant Varma, János Kramár, Anca Dragan, Rohin Shah, and Neel Nanda. 2024. [Gemma scope: Open sparse autoencoders everywhere all at once on gemma 2](#). Preprint, arXiv:2408.05147.
- Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2023. Let’s verify step by step. In *The Twelfth International Conference on Learning Representations*.
- Johnny Lin. 2023. [Neuronpedia: Interactive reference and tooling for analyzing neural networks](#). Software available from neuronpedia.org.
- Stephanie Lin, Jacob Hilton, and Owain Evans. 2021. Truthfulqa: Measuring how models mimic human falsehoods. *arXiv preprint arXiv:2109.07958*.
- Zhengkai Lin, Zhihang Fu, Ze Chen, Chao Chen, Liang Xie, Wenxiao Wang, Deng Cai, Zheng Wang, and Jieping Ye. 2025. Controlling thinking speed in reasoning models. *arXiv preprint arXiv:2507.03704*.

- Sheng Liu, Tianlang Chen, Pan Lu, Haotian Ye, Yizheng Chen, Lei Xing, and James Zou. 2025. Fractional reasoning via latent steering vectors improves inference time compute. *arXiv preprint arXiv:2506.15882*.
- Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. 2022a. Locating and editing factual associations in gpt. *Advances in neural information processing systems*, 35:17359–17372.
- Kevin Meng, Arnab Sen Sharma, Alex Andonian, Yonatan Belinkov, and David Bau. 2022b. Mass-editing memory in a transformer. *arXiv preprint arXiv:2210.07229*.
- Erik Miehl, Karthikeyan Natesan Ramamurthy, Praveen Venkateswaran, Ching-Yun Ko, Pierre Dognin, Moninder Singh, Tejaswini Pedapati, Avinash Balakrishnan, Matthew Riemer, Dennis Wei, and 1 others. 2026. Ai steerability 360: A toolkit for steering large language models. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, pages 436–444.
- Tomáš Mikolov, Wen-tau Yih, and Geoffrey Zweig. 2013. Linguistic regularities in continuous space word representations. In *Proceedings of the 2013 conference of the north american chapter of the association for computational linguistics: Human language technologies*, pages 746–751.
- Matthew Lyle Olson, Neale Ratzlaff, Musashi Hinck, Shao-yen Tseng, and Vasudev Lal. 2024. Steering large language models to evaluate and amplify creativity. *arXiv preprint arXiv:2412.06060*.
- Kiho Park, Yo Joong Choe, and Victor Veitch. 2023. The linear representation hypothesis and the geometry of large language models. *arXiv preprint arXiv:2311.03658*.
- Seongheon Park, Xuefeng Du, Min-Hsuan Yeh, Haobo Wang, and Yixuan Li. 2025. Steer llm latents for hallucination detection. *arXiv preprint arXiv:2503.01917*.
- Pranab Sahoo, Ayush Kumar Singh, Sriparna Saha, Vinija Jain, Samrat Mondal, and Aman Chadha. 2024. A systematic survey of prompt engineering in large language models: Techniques and applications. *arXiv preprint arXiv:2402.07927*.
- Marco Scialanga, Thibault Laugel, Vincent Grari, and Marcin Detyniecki. 2025. Sake: Steering activations for knowledge editing. *arXiv preprint arXiv:2503.01751*.
- Thibault Sellam, Dipanjan Das, and Ankur P Parikh. 2020. Bleurt: Learning robust metrics for text generation. *arXiv preprint arXiv:2004.04696*.
- Atakan Seyitoğlu, Aleksei Kuvshinov, Leo Schwinn, and Stephan Günnemann. 2024. Extracting unlearned information from llms with activation steering. *arXiv preprint arXiv:2411.02631*.
- Alexander Matt Turner, Lisa Thiergart, Gavin Leech, David Udell, Juan J Vazquez, Ulisse Mini, and Monte MacDiarmid. 2023. Steering language models with activation engineering. *arXiv preprint arXiv:2308.10248*.
- Theia Vogel. 2024. [repen](#).
- Junfei Wu, Yue Ding, Guofan Liu, Tianze Xia, Ziyue Huang, Dianbo Sui, Qiang Liu, Shu Wu, Liang Wang, and Tieniu Tan. 2025. Sharp: Steering hallucination in llms via representation engineering. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 14357–14372.
- Zhengxuan Wu, Aryaman Arora, Zheng Wang, Atticus Geiger, Dan Jurafsky, Christopher D Manning, and Christopher Potts. 2024. Reft: Representation finetuning for language models. *Advances in Neural Information Processing Systems*, 37:63908–63962.
- Haolei Xu, Haiwen Hong, Hongxing Li, Rui Zhou, Yang Zhang, Longtao Huang, Hui Xue, Yongliang Shen, Weiming Lu, and Yueting Zhuang. 2026. Seeing but not thinking: Routing distraction in multimodal mixture-of-experts. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 31164–31178.
- Haolei Xu, Yuchen Yan, Yongliang Shen, Wenqi Zhang, Guiyang Hou, Shengpei Jiang, Kaitao Song, Weiming Lu, Jun Xiao, and Yueting Zhuang. 2025a. Mind the gap: Bridging thought leap for improved chain-of-thought tuning. *arXiv preprint arXiv:2505.14684*.
- Ziwen Xu, Shuxun Wang, Kewei Xu, Haoming Xu, Mengru Wang, Xinle Deng, Yunzhi Yao, Guozhou Zheng, Huajun Chen, and Ningyu Zhang. 2025b. Easyedit2: An easy-to-use steering framework for editing large language models. *arXiv preprint arXiv:2504.15133*.
- An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, and 22 others. 2024. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*.
- Yifan Yao, Jinhao Duan, Kaidi Xu, Yuanfang Cai, Zhibo Sun, and Yue Zhang. 2024. A survey on large language model (llm) security and privacy: The good, the bad, and the ugly. *High-Confidence Computing*, 4(2):100211.
- Hengyuan Zhang, Zhihao Zhang, Mingyang Wang, Zunhai Su, Yiwei Wang, Qianli Wang, Shuzhou Yuan, Ercong Nie, Xufeng Duan, Qibo Xue, and 1 others. 2026a. Locate, steer, and improve: A practical survey of actionable mechanistic interpretability in large language models. *arXiv preprint arXiv:2601.14004*.
- Ningyu Zhang, Yunzhi Yao, Jiaxin Qin, Haoming Xu, Yuqi Zhu, Zeping Yu, Mengru Wang, Yuqi Tang,

Jia-Chen Gu, Shumin Deng, and 1 others. 2026b. Towards principled knowledge editing methods for large language model reasoning. *Nature Machine Intelligence*, pages 1–12.

Ningyu Zhang, Yunzhi Yao, Bozhong Tian, Peng Wang, Shumin Deng, Mengru Wang, Zekun Xi, Shengyu Mao, Jintian Zhang, Yuansheng Ni, and 1 others. 2024. A comprehensive study of knowledge editing for large language models. *arXiv preprint arXiv:2401.01286*.

Yu Zhang, Jinlong Ma, Yongshuai Hou, Xuefeng Bai, Kehai Chen, Yang Xiang, Jun Yu, and Min Zhang. 2026c. Evaluating and steering modality preferences in multi-modal llms. In *Forty-third International Conference on Machine Learning*.

Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, and 1 others. 2023. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 1(2).

## A Concept Vector Extraction Methods

EasySteer implements several concept extraction methods commonly employed in analysis-based steering. These methods identify and isolate direction vectors corresponding to specific semantic concepts within model hidden states. The fundamental approach leverages contrastive analysis on paired datasets: a positive dataset  $D^+$  containing samples exhibiting the target concept, and a negative dataset  $D^-$  containing samples that are either opposite to or independent of the target concept. By contrasting differences in model internal activations between these sample types, we can isolate the target concept’s directional representation. The extracted concept vector  $v$  is subsequently utilized in the steering intervention process described in Section 3.1.

### A.1 Contrastive Activation Addition (CAA)

CAA represents one of the most straightforward concept extraction approaches. The concept vector  $v$  is computed as the difference between mean hidden states of positive and negative sample sets:

$$v = \mathbb{E}_{x^+ \sim D^+} [h_{l,i}(x^+)] - \mathbb{E}_{x^- \sim D^-} [h_{l,i}(x^-)]$$

### A.2 Principal Component Analysis (PCA)

PCA identifies directions of maximum variance in hidden state representations, capturing the most prominent structural differences in the data. We implement two variants:

**Center PCA.** For each positive-negative sample pair  $(x_k^+, x_k^-)$ , we first compute the centroid:

$$m_k = \frac{h_{l,i}(x_k^+) + h_{l,i}(x_k^-)}{2}$$

We then apply PCA to the set of centered vectors to extract the principal direction with maximum variance:

$$v = \text{PCA}(\{h_{l,i}(x^+) - m_k, h_{l,i}(x^-) - m_k\})$$

**Diff PCA.** This variant directly applies PCA to the difference vectors between paired samples:

$$v = \text{PCA}(\{h_{l,i}(x^+) - h_{l,i}(x^-)\})$$

Since PCA produces undirected components (both  $v$  and  $-v$  are valid), we perform direction alignment by computing average projections:

$$\text{proj}^+ = \mathbb{E}_{x^+ \sim D^+} \left[ \frac{h_{l,i}(x^+)^T v}{\|v\|} \right]$$

$$\text{proj}^- = \mathbb{E}_{x^- \sim D^-} \left[ \frac{h_{l,i}(x^-)^T v}{\|v\|} \right]$$

If  $\text{proj}^+ < \text{proj}^-$ , we correct the direction:  $v \leftarrow -v$ .

### A.3 Linear Probing

Linear probing learns a linear classifier  $w \in \mathbb{R}^d$  that distinguishes between positive and negative samples through projection in hidden state space. We optimize the following objective with binary cross-entropy loss and regularization:

$$v = \arg \min_w \mathbb{E}_{x \sim D^+ \cup D^-} [\mathcal{L}_{\text{BCE}}(y_x, \sigma(w^T h_{l,i}(x)))]$$

where  $y_x \in \{0, 1\}$  denotes the class label,  $\sigma(\cdot)$  is the sigmoid function.

### A.4 Sparse Autoencoders (SAE)

Sparse autoencoders decompose hidden states  $h \in \mathbb{R}^d$  into sparse, interpretable feature representations. The architecture comprises:

**Encoder.** Maps hidden states to higher dimensional sparse activations  $f \in \mathbb{R}^n$  (where  $n \gg d$ ):

$$f = \text{ReLU}(W_{\text{enc}} h + b_{\text{enc}})$$

**Decoder.** Reconstructs the original hidden state from sparse activations:

$$\hat{h} = W_{\text{dec}} f + b_{\text{dec}}$$

Pre-trained SAEs (e.g., gemma-scope (Lieberum et al., 2024)) typically include automated interpretability analyses and semantic labels for discovered features. EasySteer provides a streamlined pipeline for extracting concept vectors from pre-trained SAEs:

1. **Concept Definition and Retrieval.** Users define target concepts in natural language and perform semantic search over pre-computed feature interpretations to identify relevant features.
2. **Feature Selection.** Based on search results and automated interpretations, users select the feature index  $k$  that best aligns with their intended concept.
3. **Vector Extraction.** The concept vector corresponds to the  $k$ -th column of the decoder weight matrix:

$$v = W_{\text{dec}}[:, k]$$

## B Learning-based Steering Methods

This appendix presents representative learning-based steering methods implemented in EasySteer. These methods follow the framework outlined in Section 3.2, optimizing parameterized steering functions  $f_\theta$  while maintaining frozen language model parameters  $W$ .

### B.1 Supervised Additive Vector

The simplest learning-based approach directly optimizes an additive steering vector  $b \in \mathbb{R}^d$  on task-specific data  $D$ :

$$f_\theta(h_{l,i}) := h_{l,i} + b$$

This method provides a baseline for more sophisticated steering techniques while maintaining computational efficiency.

### B.2 LM-Steer

LM-Steer introduces a learnable linear transformation at the model’s final layer to modulate generation behavior:

$$f_\theta(h_{l,i}) := h_{l,i} + \epsilon W h_{l,i}$$

where  $\theta = \{W\}$  with  $W \in \mathbb{R}^{d \times d}$ , and  $\epsilon$  controls the intervention strength. This approach enables more expressive steering while maintaining linearity in the transformation.

### B.3 Low-rank Linear Subspace ReFT (LoReFT)

LoReFT represents a parameter-efficient representation fine-tuning method that constrains hidden state modifications to a learned low-rank subspace. With only  $2rd + r$  parameters, it achieves effective control through:

$$f_\theta(h_{l,i}) := h_{l,i} + R^T(W h_{l,i} + b - R h_{l,i})$$

where  $\theta = \{R, W, b\}$ ,  $R \in \mathbb{R}^{r \times d}$  is the low-rank projection matrix,  $W \in \mathbb{R}^{r \times d}$  is the linear projection matrix, and  $b \in \mathbb{R}^r$  is the bias vector. The low-rank constraint ( $r \ll d$ ) ensures parameter efficiency while the learned subspace provides sufficient expressiveness for diverse steering objectives.

## C Overthinking Mitigation

### C.1 Experiment Setup

Following SEAL, we extract Execution, Reflection, and Transition vectors from 1,000 MATH training

samples and enhance Execution while suppressing the others at reasoning-step boundaries (Chen et al., 2025b). Figure 4 shows the implementation. We evaluate DeepSeek-R1-Distill-Qwen-1.5B/7B on GSM8K (Cobbe et al., 2021) and MATH500 (Lightman et al., 2023) with an 8192-token limit, measuring accuracy (Xu et al., 2025a) and token efficiency.

```
from vllm import LLM, SamplingParams
from vllm.steer_vectors.request import SteerVectorRequest, VectorConfig
from transformers import AutoTokenizer

model_name = "DeepSeek-R1-Distill-Qwen-1.5B"
tokenizer = AutoTokenizer.from_pretrained(model_name)
# Initialize LLM with steering vector capability
llm = LLM(model=model_name, enable_steer_vector=True, enforce_eager=True)
# search target token
target_token = "CC"
vocab = tokenizer.get_vocab()
target_token_id = [vocab.get(target_token)]
# Configure steering vector request for SEAL control
sv_request = SteerVectorRequest(
    steer_vector_name="seal",
    steer_vector_id=1,
    vector_configs=[
        VectorConfig(
            path="execution.gguf", scale=0.5, target_layers=[20],
            generate_trigger_tokens=target_token_id, algorithm="direct",
        ),
        VectorConfig(
            path="reflection.gguf", scale=-0.5, target_layers=[20],
            generate_trigger_tokens=target_token_id, algorithm="direct",
        ),
        VectorConfig(
            path="transition.gguf", scale=-0.5, target_layers=[20],
            generate_trigger_tokens=target_token_id, algorithm="direct",
        ),
    ],
    conflict_resolution="sequential"
)
# generate
example_answers = llm.generate(
    texts,
    SamplingParams(
        temperature=0,
        max_tokens=8192,
    ),
    steer_vector_request=sv_request
)
```

Figure 4: An illustrative code snippet of the SEAL algorithm implemented using EasySteer. Multiple steering vectors are applied to the `\n\n` token via the multi-vector collaboration functionality.

### C.2 Results and Analysis

Table 5 demonstrates that steering effectively reduces redundant reasoning steps while maintaining solution quality using EasySteer. On DeepSeek-R1-Distill-Qwen-1.5B, SEAL improves GSM8K accuracy by 2.7% (79.6%→82.3%) while reducing token usage by 40.0%. The 7B model shows similar efficiency gains with 13.3% and 16.8% token reduction on GSM8K and MATH500 respectively.

| Model                         | Method | GSM8K       |                | MATH500     |                |
|-------------------------------|--------|-------------|----------------|-------------|----------------|
|                               |        | Acc↑        | Tokens↓        | Acc↑        | Tokens↓        |
| DeepSeek-R1-Distill-Qwen-1.5B | /      | 79.6        | 2435.48        | 70.8        | 3966.38        |
|                               | SEAL   | <b>82.3</b> | <b>1460.13</b> | <b>78.4</b> | <b>3074.67</b> |
| DeepSeek-R1-Distill-Qwen-7B   | /      | <b>90.3</b> | 792.74         | 86.6        | 3096.69        |
|                               | SEAL   | 88.5        | <b>687.52</b>  | <b>88.2</b> | <b>2577.04</b> |

Table 5: Overthinking mitigation performance using SEAL steering.